ADITYA UNIVERSITY

School of Computing
Department of Information Technology

PROJECT REPORT
ON

CREDIT CARD FRAUD DETECTION SYSTEM

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
in
Information Technology

Submitted by
SAI HARINI SINGINEEDI (24B11IT097)
NALLI PRANEESHA (24B11IT072)
PEDAPUDI SNEHALATHA (24B11IT083)
MURALASETTI ANANTHA LAKSHMI (25B21IT007)

Project Guide
Mrs. J. Divya Lalitha Sri
Department of Information Technology

Academic Year: 2026–2027

ADITYA UNIVERSITY

School of Computing

Department of Information Technology

CERTIFICATE

This is to certify that the project work entitled **"Credit Card Fraud Detection System"** is submitted by

SAI HARINI SINGINEEDI (24B11IT097)
NALLI PRANEESHA (24B11IT072)
PEDAPUDI SNEHALATHA (24B11IT083)
MURALASETTI ANANTHA LAKSHMI (25B21IT007)

in partial fulfilment of the requirements for the award of the B.Tech. degree in Information Technology for the academic year 2026–2027.

Project Guide
Mrs. J. Divya Lalitha Sri
Department of Information Technology

Head of the Department
Dr. M. Raja Babu
Department of Information Technology

DECLARATION

We hereby declare that the project entitled **"Credit Card Fraud Detection System"** is original and is submitted to Aditya University, Surampalem in partial fulfilment of the requirements for the award of the B.Tech. degree in Information Technology. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy.

Place: Surampalem

Date:

By

SAI HARINI SINGINEEDI (24B11IT097)

NALLI PRANEESHA (24B11IT072)

PEDAPUDI SNEHALATHA (24B11IT083)

MURALASETTI ANANTHA LAKSHMI (25B21IT007)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Project Guide, Mrs. J. Divya Lalitha Sri, who has guided us throughout the project. Their valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We are grateful to Dr. M. Raja Babu, Head of the Department, for inspiring us all the way and for arranging all the facilities and resources needed for our project. We wish to thank Dr. M. V. Rajesh, Associate Dean (School of Computing), Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), and Dr. G. Suresh, Registrar, for their encouragement and support during the course of our project.

We would like to extend our sincere thanks to Dr. M. B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor, and the Management, Aditya University, for their unconditional support in providing us with the best infrastructural facilities and state-of-the-art laboratories during our project.

Not to forget, the non-teaching staff and our friends, who have directly or indirectly supported us in completing this project on time.

ABSTRACT

Credit card fraud causes substantial financial loss, yet fraudulent transactions are hard to detect because they form only a tiny fraction of all card activity and are designed to resemble ordinary purchases. This project develops a machine-learning-based credit card fraud detection prototype using the fraudTrain.csv and fraudTest.csv transaction files, which contain 1,296,675 training and 555,719 testing transactions and are severely imbalanced (7,506 fraudulent against 1,289,169 legitimate training transactions). After unused identifier columns were removed, the data was preprocessed and enriched with transaction-time, geographic, and behavioral features derived from each card's earlier activity, giving 18 model input features, and the models were evaluated on a later chronological period than the one used for training. Logistic Regression, Random Forest, and XGBoost were compared using precision, recall, F1-score, ROC-AUC, and PR-AUC. At the default threshold of 0.50, XGBoost achieved the highest PR-AUC (0.8537) and F1-score (0.7980) and was therefore selected; optimizing the classification threshold to 0.25, the best F1-score among the thresholds tested, raised the F1-score to 80.94% with a precision of 91.53% and a recall of 72.54%. SHAP is used to explain how individual features contribute to the model's output, and the prediction engine is integrated into a FastAPI backend and a React frontend that display the fraud probability, classification, risk level, and explanation. The project contributes an explainable, end-to-end fraud-risk estimation prototype; its limitations are that it was trained and evaluated on a single dataset, is not connected to any live banking system, and produces risk estimates that are not proof of fraud.

Keywords: Machine Learning, Credit Card Fraud Detection, XGBoost, Class Imbalance, Feature Engineering, SHAP, Threshold Optimization, Model Evaluation, FastAPI, React

LIST OF CONTENTS

LIST OF FIGURES....vi
Chapter 1: Introduction....1
1.1 Introduction....1
1.2 Problem Statement....1
1.3 Objectives of the Project....2
1.4 Scope of the Project....2
Chapter 2: System Overview....3
2.1 Existing System....3
2.2 Limitations of Existing System....3
2.3 Proposed System....3
2.4 Software Requirements....4
2.5 Hardware Requirements....4
Chapter 3: System Implementation....6
3.1 Overview of Development Tools....6
3.2 Dataset Description....7
Chapter 4: System Architecture....9
Chapter 5: Data Preprocessing....12
5.1 Introduction....12
5.2 Handling Missing Values....12
5.3 Data Cleaning....12
5.4 Outlier Detection and Range Validation....12
5.5 Feature Engineering / Encoding....13
5.6 Data Visualization....14
5.7 Train-Test Split....15
5.8 Summary....15
Chapter 6: Methodology and Algorithms....16
6.1 Introduction....16
6.2 Classification Techniques....16
6.3 Ensemble Learning — XGBoost....17
6.4 SHAP Explainability and Threshold Optimization....17
6.5 Model Evaluation Metrics....18
Chapter 7: Results and Analysis....20
7.1 Introduction....20
7.2 Experimental Setup....20
7.3 Logistic Regression Results....20
7.4 Random Forest and XGBoost Results....20
7.5 Comparison of Models....21
7.6 Key Findings....22
Chapter 8: Output Screens....24
Chapter 9: Conclusion....29
Chapter 10: Future Scope....30
Chapter 11: References....31

LIST OF FIGURES

Fig 2.1: Overall Project Workflow....4
Fig 3.1: Development Environment and Project Folder....7
Fig 3.2: Training and Testing Dataset Class Distribution....8
Fig 4.1: System Architecture....9
Fig 4.2: Prediction Flow for a Single Transaction....10
Fig 5.1: Feature Engineering Workflow....13
Fig 5.2: Transaction Amount Distribution....14
Fig 5.3: XGBoost Feature Importance....15
Fig 7.1: Model Performance Comparison....21
Fig 7.2: Threshold Optimization Results....22
Fig 7.3: SHAP Summary Plot (Global Feature Importance)....23
Fig 8.1: Project Directory and Development Environment....24
Fig 8.2: Dataset Analysis and Class Distribution Output....24
Fig 8.3: Data Preprocessing Output....25
Fig 8.4: Model Training Output....25
Fig 8.5: Model Comparison Table or Chart....26
Fig 8.6: Threshold Optimization Results....26
Fig 8.7: SHAP Feature Importance Plot....26
Fig 8.8: Transaction Input Interface....27
Fig 8.9: Legitimate Sample Prediction....27
Fig 8.10: Fraud Sample Prediction....27
Fig 8.11: Fraud Probability and Risk Level....28
Fig 8.12: Explanation and Contributing Factors....28

CHAPTER 1

INTRODUCTION

1.1 Introduction

Credit cards are among the most widely used electronic payment instruments, and the growth of online and card-not-present payments has increased the opportunities for fraudulent use. Credit card fraud occurs when a card or its details are used without the cardholder's authorization. It causes financial loss to cardholders and card issuers and weakens trust in digital payments, which makes fast and reliable detection an important problem for financial institutions.

Fraudulent transactions are difficult to identify for several reasons. They are rare compared with legitimate transactions, they are deliberately made to resemble ordinary purchases, and the behavior of both customers and fraudsters changes over time. A single attribute, such as a large amount, is rarely enough to separate fraud from genuine activity. The evidence usually lies in combinations of amount, merchant category, time of day, location, and the way a transaction differs from the earlier behavior of the same card. Supervised machine learning is well suited to this setting because it can learn such patterns automatically from historical transactions that are already labeled as legitimate or fraudulent, and can then estimate how likely a new transaction is to be fraudulent.

This project presents a **Credit Card Fraud Detection System** built on the `fraudTrain.csv` and `fraudTest.csv` transaction files. The project preprocesses the transaction records, engineers transaction-time, geographic, and card-behavior features, trains and compares Logistic Regression, Random Forest, and XGBoost classifiers, and selects XGBoost as the final model. For a given transaction, the system produces a **fraud probability**, a **classification** (fraudulent or legitimate), a **risk level** (LOW, MEDIUM, or HIGH), and a **SHAP-based explanation** of the factors that influenced the estimate. The trained model is exposed through a FastAPI backend and presented through a React web interface. The output of the system is a model-based risk estimate; it is not proof that a transaction is fraudulent.

1.2 Problem Statement

The central difficulty is **class imbalance**. In the training data used in this project, only 7,506 of 1,296,675 transactions (about 0.58%) are fraudulent, and in the testing data 2,145 of 555,719 transactions (about 0.39%) are fraudulent. A model that labeled every transaction as legitimate would therefore be correct for about 99.61% of the testing transactions while detecting no fraud at all. Accuracy alone is consequently a misleading measure for this problem, and metrics that concentrate on the minority class, namely precision, recall, F1-score, and PR-AUC, are required.

A second difficulty is the trade-off between two kinds of error. Missing a fraudulent transaction (a false negative) causes direct financial loss, whereas flagging a genuine transaction (a false positive)

inconveniences the customer and consumes investigation effort. The classification threshold controls the balance between these errors and must be chosen deliberately instead of being left at a default value.

Finally, a probability or a label without a reason is of limited use to a human reviewer. An explainable prediction, one that shows which features pushed the estimate up or down, allows the result to be examined and questioned. This project therefore aims to build an imbalance-aware, properly evaluated, and explainable fraud-risk estimation system.

1.3 Objectives of the Project

- Analyze and preprocess historical credit card transaction data.
- Engineer transaction-time, geographic, and card-behavior features from the raw transaction records.
- Train and compare Logistic Regression, Random Forest, and XGBoost classifiers.
- Evaluate fraud detection performance using Precision, Recall, F1-score, ROC-AUC, and PR-AUC.
- Optimize the classification threshold of the XGBoost model.
- Use SHAP to explain individual predictions.
- Integrate the trained model into a web application using a FastAPI backend and a React frontend.

1.4 Scope of the Project

The scope of the project covers the following activities:

- Analysis of the historical transaction datasets (`fraudTrain.csv` and `fraudTest.csv`).
- Data preprocessing and behavioral feature engineering.
- Training and evaluation of Logistic Regression, Random Forest, and XGBoost models.
- Threshold optimization for the selected XGBoost model.
- SHAP-based explainability of model outputs.
- A prediction engine that uses the saved model artifacts.
- A FastAPI backend (served with Uvicorn) and a React user interface that display the fraud probability, classification, risk level, and explanation.

Out of scope. Live integration with bank or card-network systems and the blocking or declining of real transactions are outside the current implementation. The application is a prototype that estimates fraud risk for transaction details supplied to it; it does not act on those estimates.

CHAPTER 2

SYSTEM OVERVIEW

2.1 Existing System

This section gives general background on how card fraud is commonly detected. It does not describe any specific bank's or vendor's system, and no particular banking product was evaluated in this project.

Conventional fraud detection has relied heavily on rule-based checks defined by analysts. Typical rules compare a transaction against fixed limits or conditions, for example an amount above a set value, several transactions in a short period, a purchase far from the cardholder's usual location, or a merchant on a watch list. Transactions that trigger a rule are blocked, challenged, or sent for manual review. Such rules are transparent and easy to understand, and they remain a useful first line of defense.

However, fraud patterns evolve. Once fraudsters learn which behavior triggers an alert, they adapt, and the rule set has to be reviewed and updated by people. Detecting fraud that is spread across many subtle signals, such as an unusual combination of amount, time, merchant category, and deviation from the card's own history, is difficult for rules that examine only a few attributes at a time.

2.2 Limitations of Existing System

The following are general challenges associated with purely rule-based or weakly explained detection approaches. They are not measured defects of any particular banking product.

- **Rigid rules:** fixed thresholds and conditions do not adapt automatically when fraud behavior changes and require manual maintenance.
- **Difficulty with complex behavioral patterns:** rules that look at one or two attributes can miss patterns that appear only when several features are considered together.
- **False alerts:** strict rules may flag legitimate but unusual purchases, which inconveniences customers and increases the review workload.
- **Missed fraud:** fraudulent activity designed to stay within known rule limits may go undetected.
- **Limited interpretability in some automated systems:** some automated scoring approaches return a score or a decision without a human-readable reason, which makes review and trust harder.

2.3 Proposed System

The proposed system replaces fixed rules with a supervised machine learning model that learns fraud patterns from labeled historical transactions. The complete workflow is shown in Fig 2.1. Historical transaction data is preprocessed and enriched with engineered features; Logistic Regression, Random Forest, and XGBoost are trained and compared; the classification threshold of the selected XGBoost model is optimized; and the final model is used by a prediction engine whose results are explained with SHAP and served through a FastAPI backend to a React interface.

How fraud risk is estimated and displayed. For a transaction submitted through the interface, the XGBoost model estimates the probability that the transaction is fraudulent. The probability is compared with the selected classification threshold of 0.25 to obtain a classification, and it is mapped to an application-defined risk level: LOW for probabilities below 0.25, MEDIUM from 0.25 up to (but not including) 0.60, and HIGH for 0.60 or above. SHAP values for that transaction are converted into a short human-readable summary of the factors that raised or lowered the estimate. The FastAPI backend returns these results to the React interface, which displays the fraud probability, classification, risk level, and explanation. The output is an estimate, not proof of fraud.

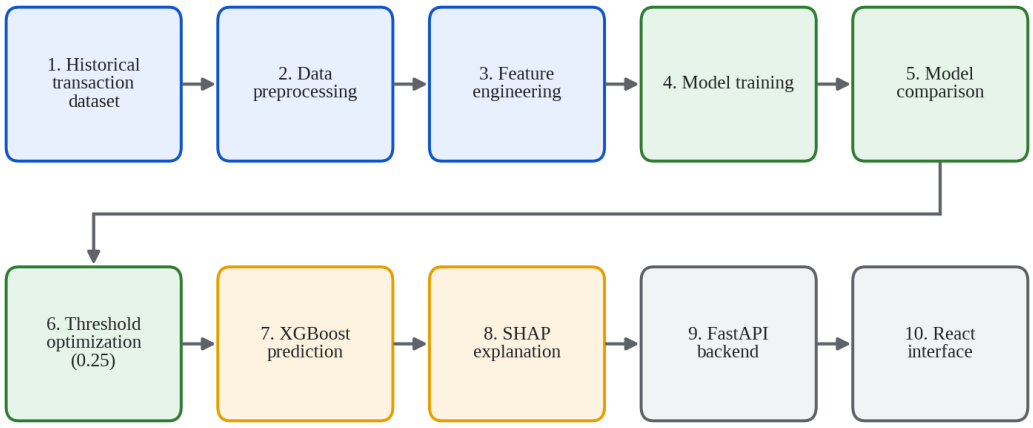


Fig 2.1: Overall Project Workflow

2.4 Software Requirements

- **Operating System:** Windows (as used in the development environment)
- **Programming Language:** Python
- **Data Handling:** Pandas and NumPy
- **Machine Learning:** Scikit-learn and XGBoost
- **Explainability:** SHAP
- **Backend:** FastAPI and Uvicorn
- **Frontend:** React, Vite, JavaScript/JSX, and CSS (running on Node.js)
- **IDE:** Visual Studio Code
- **Data Format:** CSV
- **Version Control:** Git / GitHub

2.5 Hardware Requirements

No specialized hardware is needed: a standard modern 64-bit development computer that can process roughly 1.85 million transactions and run Python and Node.js is sufficient. The values below are **recommended**, not measured or benchmarked.

- **Processor:** Modern 64-bit multi-core processor (for example, Intel Core i5 or equivalent) — recommended

- **RAM:** 8 GB or higher — recommended; more memory is helpful when processing the full datasets
- **Storage:** Several GB of free space (SSD preferred) for datasets, Python and Node.js packages, and model files — recommended
- **System Type:** 64-bit operating system

CHAPTER 3

SYSTEM IMPLEMENTATION

3.1 Overview of Development Tools

The system combines a Python-based machine learning pipeline with a web application. Table 3.1 summarizes the role of each tool in the project.

Table 3.1: Role of the Development Tools

Tool	Role in the project
Python	Main programming language for preprocessing, model training, evaluation, the prediction engine, and the backend.
Pandas	Loading the CSV files, cleaning data, grouping transactions by card, and deriving features.
NumPy	Numerical operations and array handling used during feature computation and modeling.
Scikit-learn	Logistic Regression and Random Forest classifiers and supporting machine learning utilities.
XGBoost	Gradient-boosted tree classifier; the final selected model.
SHAP	Explains how individual features contribute to the model's output.
FastAPI / Uvicorn	Backend API framework that exposes the prediction engine, and the server that runs it.
React	JavaScript library used to build the user interface.
Vite	Frontend build tool and development server for the React application.

3.1.1 Development Environment

The project was developed in **Visual Studio Code** on a Windows machine. A Python virtual environment was used to keep the project's libraries separate from other Python installations. The project is organized into separate locations for the CSV datasets, the trained model files, the backend code, and the frontend application. The trained model and its feature list are stored in the `models` folder as `xgboost.pkl` and `feature_columns.pkl`. The frontend runs on Node.js and is served during development by the Vite development server, while the backend is run with Uvicorn.

3.1.2 Machine Learning Libraries

Pandas and **NumPy** handle data loading, cleaning, and numerical computation. **Scikit-learn** provides the Logistic Regression and Random Forest models used for comparison. **XGBoost** provides the final gradient-boosted model. **SHAP** is applied to the trained XGBoost model to explain its outputs. The results of training are stored as saved model artifacts: `models/xgboost.pkl` holds the trained XGBoost model, and `models/feature_columns.pkl` holds the feature columns used by the model so that incoming transactions can be arranged in the same form as the training data. Together these files allow the prediction engine to load the model without retraining. Fig 3.1 shows the development environment and project folder.

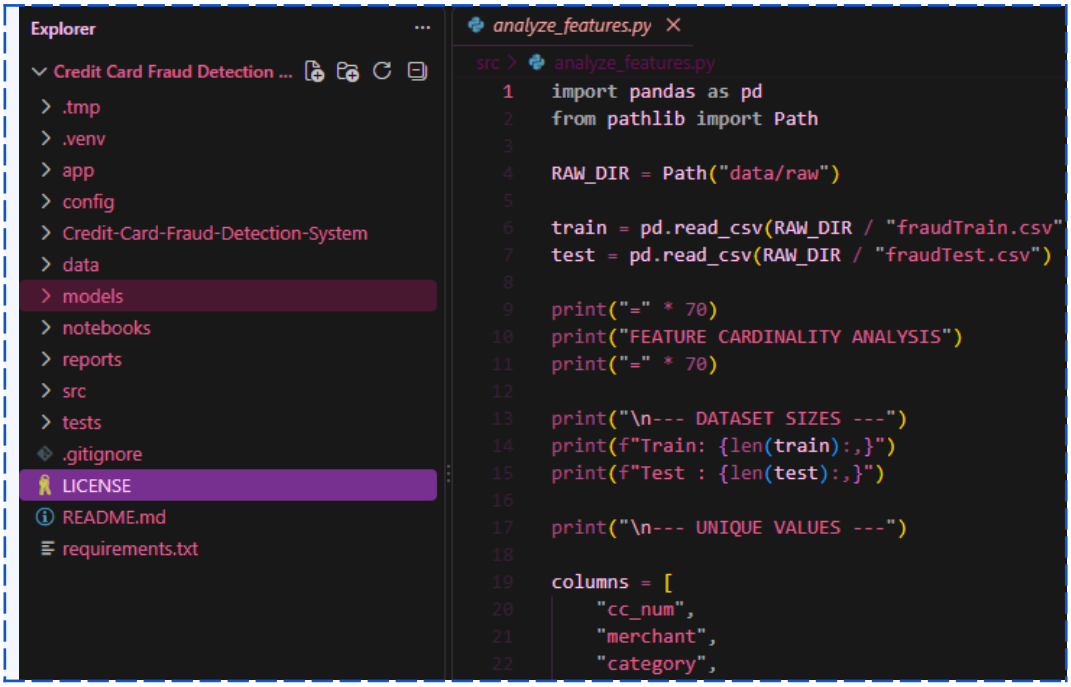


Fig 3.1: Development Environment and Project Folder

3.2 Dataset Description

The primary dataset consists of two CSV files: `fraudTrain.csv`, used for training, and `fraudTest.csv`, used for testing. Dataset source: Kartik2112, "Credit Card Transactions Fraud Detection Dataset," Kaggle. Available: https://www.kaggle.com/datasets/kartik2112/fraud-detection. [Accessed: 18-Oct-2026]. Table 3.2 summarizes the files.

Table 3.2: Dataset Description

Dataset file	Transactions	Legitimate (0)	Fraudulent (1)	Fraud share
fraudTrain.csv (training)	1,296,675	1,289,169	7,506	≈ 0.58%
fraudTest.csv (testing)	555,719	553,574	2,145	≈ 0.39%

The original records contain the transaction timestamp, merchant, transaction category, amount, gender, city, state, ZIP code, coordinates, city population, job, date of birth, card number, a transaction identifier, and the fraud label. The **target variable** is `is_fraud`, where `0` denotes a legitimate transaction and `1` denotes a fraudulent transaction.

The dataset is **severely imbalanced**: fraudulent transactions make up well under one percent of both files (Fig 3.2). In the training data the ratio of legitimate to fraudulent transactions is about 171.75 to 1, a value that is later used to set the class-imbalance parameter of XGBoost. The two files also cover **different chronological periods**: the training file covers the earlier period and the testing file the later period (Section 5.7), so the evaluation reflects performance on future transactions.

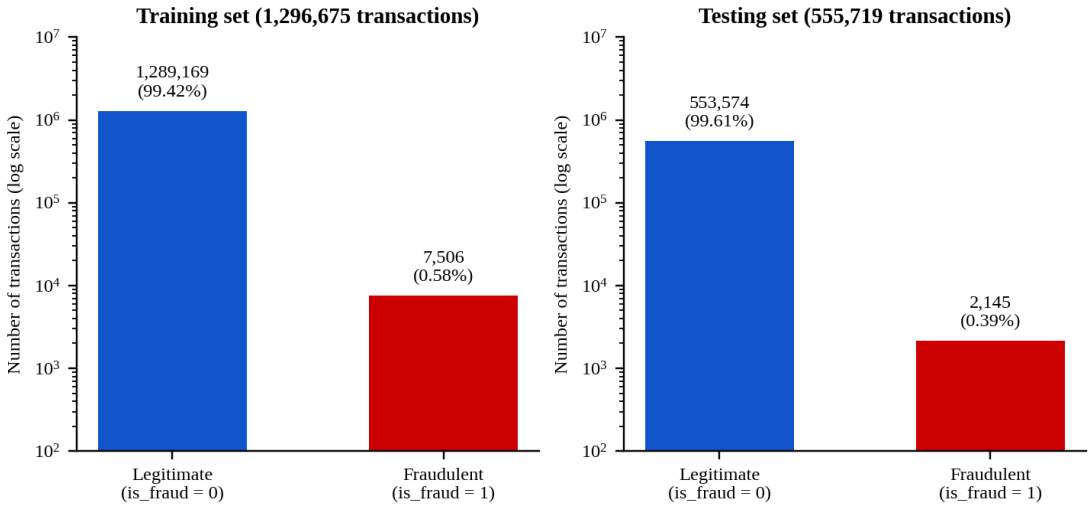


Fig 3.2: Training and Testing Dataset Class Distribution

The raw card number identifies a cardholder's card and is sensitive. It is used only as a grouping key during feature derivation (Section 5.3), and raw card numbers are not reproduced anywhere in this report or in its screenshots.

CHAPTER 4

SYSTEM ARCHITECTURE

The architecture has two parts, shown in Fig 4.1. The **offline model development** part turns the historical transaction dataset into a trained, evaluated, and saved XGBoost model. The **online prediction application** part loads the saved model artifacts in a prediction engine, which is accessed through a FastAPI backend by a React frontend. The numbers in the figure correspond to the components described in Table 4.1.

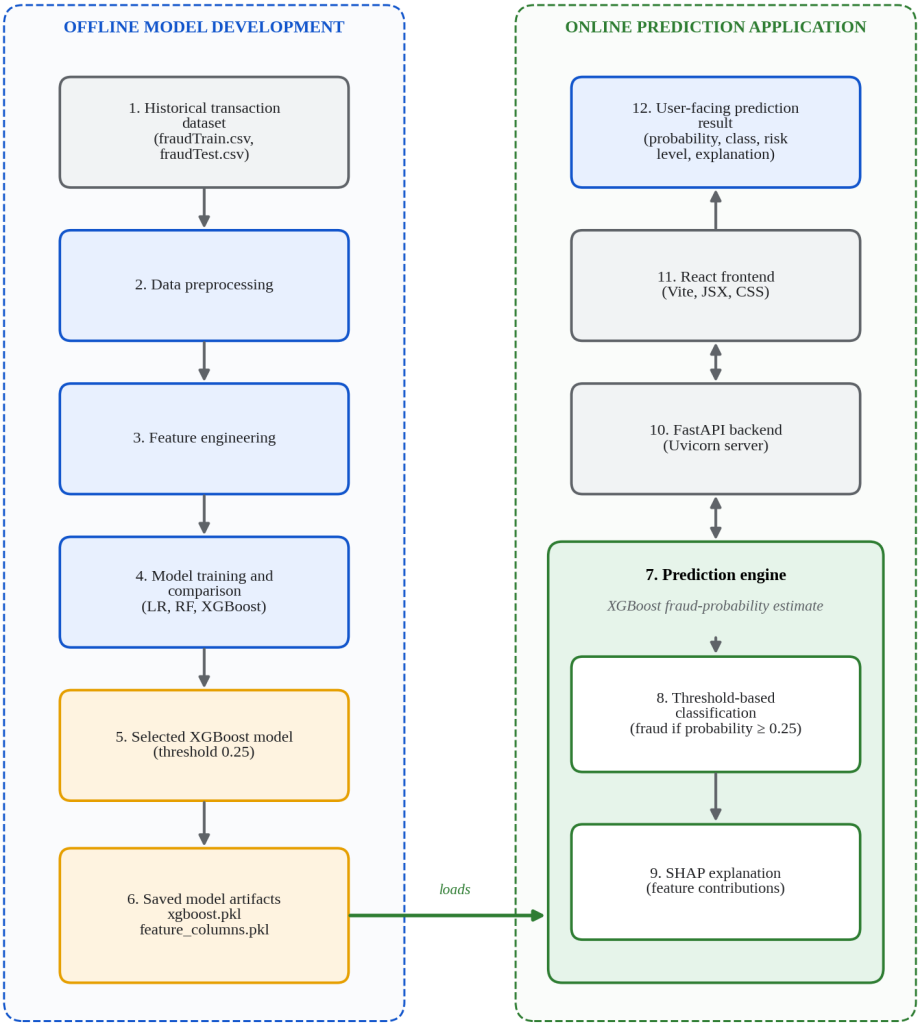


Fig 4.1: System Architecture

Table 4.1: Architecture Components

No.	Component	Description
1	Historical transaction dataset	`fraudTrain.csv` and `fraudTest.csv`, containing labeled historical card transactions.
2	Data preprocessing	Missing-value check and median filling for numeric columns, and removal of unused identifier columns.
3	Feature engineering	Derivation of time, geographic, and card-behavior features, giving 18 model input features.

No.	Component	Description
4	Model training and comparison	Logistic Regression, Random Forest, and XGBoost are trained and compared on the chronological test set.
5	Selected XGBoost model	The model with the highest PR-AUC and F1-score at the default threshold; classification threshold optimized to 0.25.
6	Saved model artifacts	`models/xgboost.pkl` and `models/feature_columns.pkl`.
7	Prediction engine	Loads the saved artifacts, prepares features for a transaction, and obtains the fraud probability.
8	Threshold-based classification	Compares the probability with the 0.25 threshold and assigns the application-defined risk level.
9	SHAP explanation	Computes feature contributions for the individual transaction.
10	FastAPI backend	Exposes the prediction engine to the frontend and returns the results (served with Uvicorn).
11	React frontend	Interface (built with Vite) for entering transaction details and viewing results.
12	User-facing prediction result	Fraud probability, classification, risk level, and explanation shown to the user.

Prediction flow. Fig 4.2 shows what happens when a user submits a transaction. The input is processed and converted into the feature form expected by the model, using the feature columns stored in `models/feature_columns.pkl`. The XGBoost model stored in `models/xgboost.pkl` estimates the fraud probability. The probability is compared with the classification threshold, a risk level is assigned, and SHAP contributions for that transaction are converted into a human-readable summary that the frontend displays.

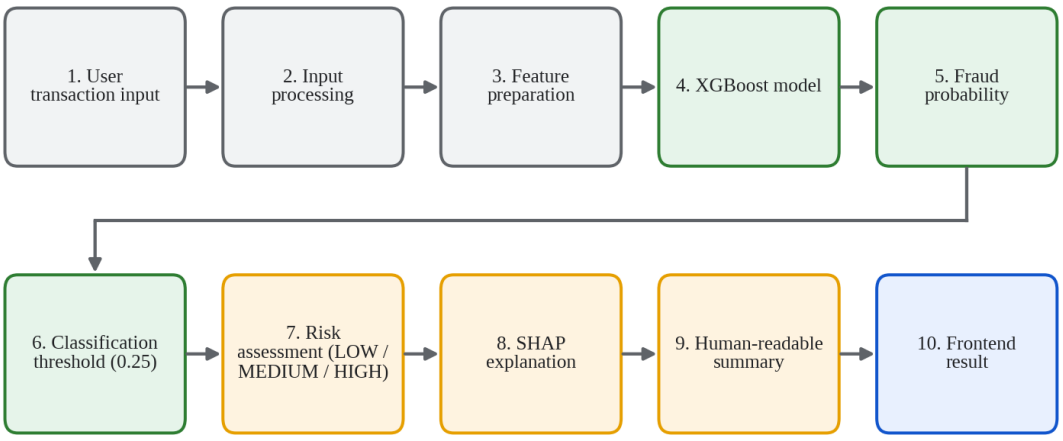


Fig 4.2: Prediction Flow for a Single Transaction

Risk levels. The risk levels are defined by the application and are not properties of the dataset or the model. They are applied to the fraud probability as shown in Table 4.2.

Table 4.2: Application-Defined Risk Levels

Fraud probability	Risk level
Below 0.25	LOW
0.25 to below 0.60	MEDIUM
0.60 or above	HIGH

Implementation boundaries. The current architecture contains no MongoDB or other database, no cloud services, and no banking APIs. The application runs in the local development environment and is not connected to any live banking system.

CHAPTER 5

DATA PREPROCESSING

5.1 Introduction

Raw transaction records cannot be given directly to a machine learning model. They contain identifiers that carry no predictive meaning, categorical text that must be converted to numbers, timestamps that must be turned into useful time features, and card histories that must be summarized. Preprocessing converts the original records into a consistent numeric form and derives features that capture behavior. In this project, **Pandas** was used to load the CSV files, clean the data, group transactions by card, and derive features, and **NumPy** supported the numerical computations.

5.2 Handling Missing Values

Missing values were checked in both processed datasets, and the observed result was **zero missing values** in both the processed training data and the processed testing data. In the implemented pipeline, numeric missing values are filled using medians computed from the training set, so that no information from the testing period influences the fill values.

5.3 Data Cleaning

Columns that do not help the model generalize were removed. These include the first name, last name, street, transaction number, the raw index column, and other unused identifiers. Removing them reduces noise and prevents the model from memorizing identities instead of learning behavior.

The raw **card number** was used only as a grouping key. Grouping the transactions of each card in time order made it possible to derive the historical features described in Section 5.5. The card number itself was not passed to the model as an input feature.

5.4 Outlier Detection and Range Validation

Numeric attributes such as the transaction amount, the distance between the cardholder and the merchant, and the behavioral features should be checked against sensible ranges, because impossible values (for example negative amounts or negative elapsed times) would indicate data errors. Unusually large amounts or behavioral values also deserve investigation. In fraud detection they are not automatically errors, since an extreme value may be exactly the signal that distinguishes a fraudulent transaction.

The processed training data showed transaction amounts (`amt`) ranging from 1.00 to 28,948.90. The engineered merchant distance ranged from approximately 0.022 to 152.1 km, while the previous transaction count ranged from 0 to 3,122. Historical average transaction amounts ranged from approximately 1.03 to 1,433.54, and the time since the previous transaction ranged from 0 to 372.63 hours. These ranges were reviewed as part of data validation. A separate IQR- or Z-score-based outlier-

removal procedure was not implemented, because extreme transaction values may contain useful fraud-related information.

5.5 Feature Engineering / Encoding

The processed data contains **18 model input features**, listed in Table 5.1 and grouped in Fig 5.1. Categorical attributes (`merchant`, `category`, `gender`, and `state`) are used in encoded numeric form. These columns were encoded using pandas categorical codes (`.cat.codes`).

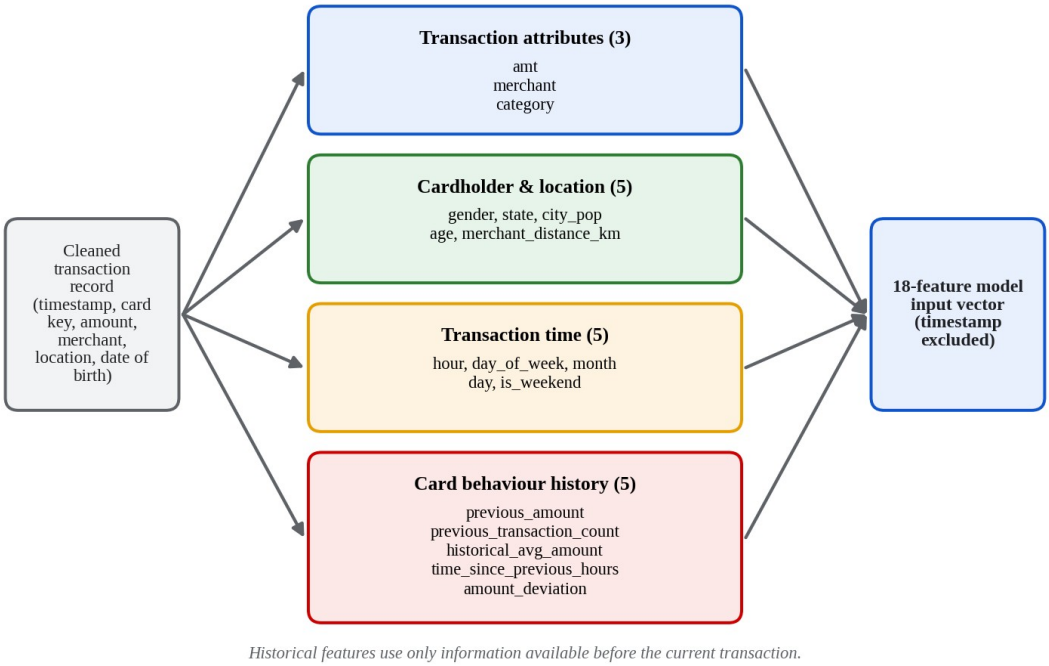


Fig 5.1: Feature Engineering Workflow

Table 5.1: Engineered Feature Description

Feature	Description
`amt`	Transaction amount.
`merchant`	Encoded merchant category/identity.
`category`	Encoded transaction category.
`gender`	Encoded gender of the cardholder.
`state`	Encoded state.
`city_pop`	Population associated with the transaction location.
`hour`	Hour of the transaction.
`day_of_week`	Day of the week.
`month`	Month of the transaction.
`day`	Day of the month.
`is_weekend`	Weekend indicator.
`age`	Age derived from the date of birth.
`merchant_distance_km`	Calculated geographic distance in kilometers.
`previous_amount`	Amount of the previous transaction on the same card.
`previous_transaction_count`	Number of earlier transactions on the card.

Feature	Description
`historical_avg_amount`	Historical average transaction amount of the card.
`time_since_previous_hours`	Elapsed time in hours since the card's previous transaction.
`amount_deviation`	Difference between the current amount and the card's historical spending behavior.

Avoiding future-data leakage. The card-behavior features (`previous_amount`, `previous_transaction_count`, `historical_avg_amount`, `time_since_previous_hours`, and `amount_deviation`) must be computed using **only information available before the transaction** being described. If a feature were allowed to use later transactions, the model would learn from information that would not exist when a real transaction is scored, and its measured performance would be unrealistically high.

Processed data. The processed training data has shape 1,296,675 × 20 and the processed testing data has shape 555,719 × 20. Each file contains the 18 model input features, the target label (`is_fraud`), and the timestamp column used during processing. The timestamp is excluded from the final model input.

5.6 Data Visualization

Several plots support understanding of the data and the results. The class distribution of the training and testing sets is shown in Fig 3.2 and makes the imbalance visible. The distribution of transaction amounts (Fig 5.2) helps to see the typical range of values and the presence of very large amounts, and the XGBoost feature importance (Fig 5.3) shows which features the model relies on most. The comparison of the three models is presented in Chapter 7 (Fig 7.1), and the SHAP summary plot is presented in Section 7.6 (Fig 7.3). Figures 5.2 and 5.3 are placeholders for plots generated from the project's own code.

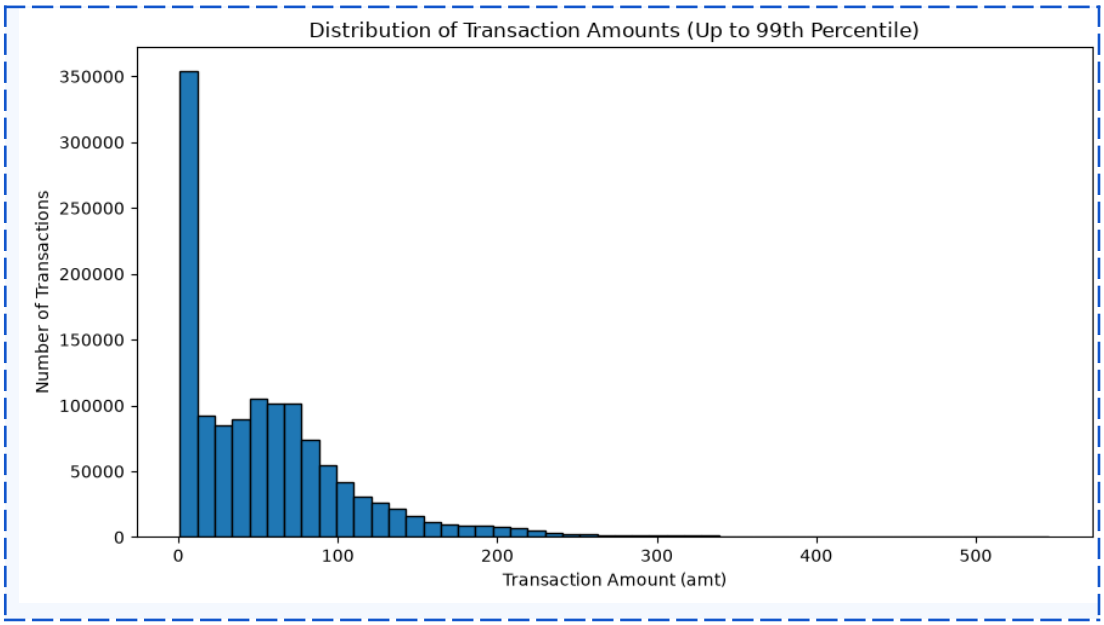


Fig 5.2: Transaction Amount Distribution

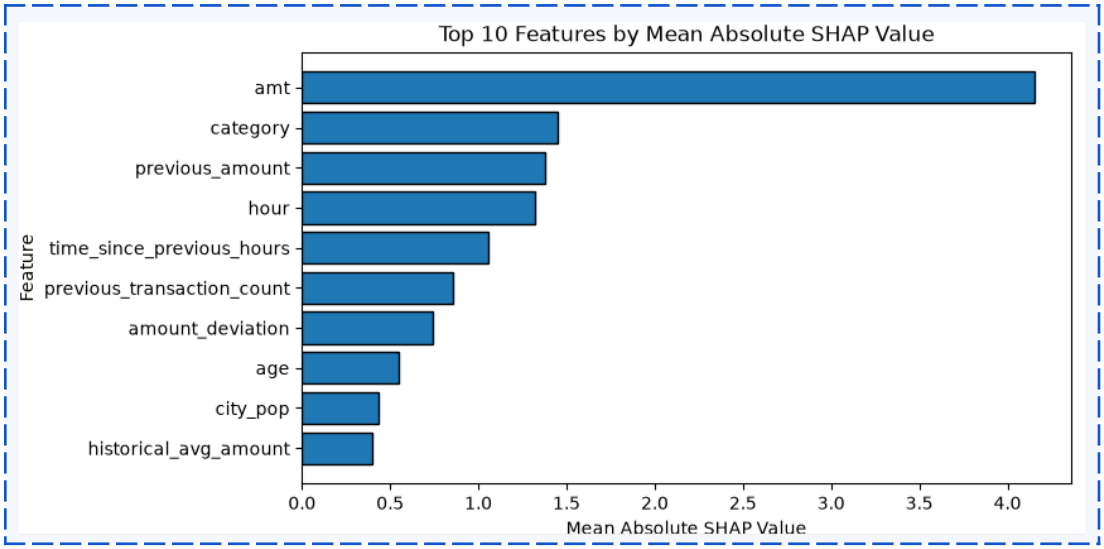


Fig 5.3: XGBoost Feature Importance

5.7 Train-Test Split

The data is separated **chronologically**. The training period runs from January 1, 2019 to June 21, 2020, and the testing period runs from June 21, 2020 to December 31, 2020. The model is trained only on the earlier transactions and evaluated on the later ones.

This design suits fraud detection because a deployed model always scores transactions that occur after the ones it was trained on. Evaluating on later transactions therefore estimates how the model would perform on future data and, together with the leakage-free historical features, avoids overly optimistic results. A random 80/20 split, a stratified split, and cross-validation were not used in this project.

5.8 Summary

Preprocessing transformed the original transaction records into model-ready data. Unused identifier columns were removed, the raw card number was used only as a grouping key, and numeric missing values were handled with training-set medians, leaving no missing values. Behavioral features were derived from each card's earlier transactions, together with time, geographic, and demographic features, giving 18 input features. The chronological separation of the training and testing periods was preserved so that the evaluation reflects realistic future performance.

CHAPTER 6

METHODOLOGY AND ALGORITHMS

6.1 Introduction

Credit card fraud detection in this project is a **supervised binary classification** problem. Each historical transaction carries a label, `is_fraud`, that marks it as legitimate (0) or fraudulent (1), and the models learn to predict that label from the 18 input features. Three classifiers were trained and compared, and two additional methods, threshold optimization and SHAP explainability, were applied to the selected model.

6.2 Classification Techniques

6.2.1 Logistic Regression

Working: Logistic Regression models the probability of a binary outcome as a logistic (sigmoid) function of a weighted sum of the input features, as given in Equation (6.1). The weights are learned from the training data.

$$P(y=1 \mid x) = \frac{1}{1 + e^{-(w^T x + b)}} \tag{6.1}$$

Advantages: it is simple, fast to train, and easy to interpret, and it provides a natural baseline against which more complex models can be compared.

Application in Project: Logistic Regression served as the baseline model. Class weighting was used to give more importance to the rare fraudulent class, and feature scaling was applied because the model is sensitive to the scale of its inputs.

6.2.2 Random Forest

Working: Random Forest builds many decision trees, each trained on a random sample of the data and a random subset of features, and combines their outputs. Because it combines many trees, it can capture nonlinear relationships and interactions between features and is less prone to overfitting than a single tree.

Advantages: it handles nonlinear relationships and mixed feature types, needs little feature scaling, and is relatively robust.

Application in Project: a Random Forest classifier was trained with balanced class weights so that errors on the minority fraud class carry more weight during training.

6.3 Ensemble Learning — XGBoost

6.3.1 XGBoost (Extreme Gradient Boosting)

Working: XGBoost is an implementation of gradient boosting [2]. It builds decision trees **sequentially**, where each new tree is trained to correct the errors made by the trees before it. The prediction is the sum of the outputs of all trees, as in Equation (6.2), and each new tree is added with a learning rate η, as in Equation (6.3).

$$\hat{y}_i = \sum_{k=1}^{K} f_k(x_i) \tag{6.2}$$

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x) \tag{6.3}$$

Advantages: it models nonlinear relationships and feature interactions, works efficiently on large tabular datasets, includes regularization to limit overfitting, and provides a parameter for handling class imbalance.

Application in Project: XGBoost is the final model. To compensate for the imbalance between classes, the parameter `scale_pos_weight` was set to the ratio of legitimate to fraudulent training transactions, as shown in Equation (6.4).

$$\text{scale_pos_weight} = \frac{N_{\text{legitimate}}}{N_{\text{fraud}}} = \frac{1{,}289{,}169}{7{,}506} \approx 171.75 \tag{6.4}$$

XGBoost was selected because it achieved the **highest PR-AUC and F1-score** among the three tested models at the reported default threshold of 0.50 (Chapter 7).

6.4 SHAP Explainability and Threshold Optimization

Two additional methods were applied to the selected XGBoost model: SHAP for explainability and threshold optimization for choosing the operating point.

6.4.1 SHAP Explainability

Working: SHAP (SHapley Additive exPlanations) [3] estimates how much each feature contributes to a model output, based on Shapley values from cooperative game theory. The contributions of all features, added to a base value, account for the model's output for a transaction. SHAP can be viewed at two levels. A **global** view summarizes which features matter most across many transactions, and a **local** view explains the output for one individual transaction. These two views are kept separate in this report.

Application in Project: SHAP is applied to the trained XGBoost model. The global SHAP summary is used to understand which features influence the model overall, and the local SHAP contributions for a

submitted transaction are used to build the explanation shown in the web application. SHAP explains the **model**; it does not independently determine whether a transaction is fraudulent.

6.4.2 Threshold Optimization

Working: a classifier outputs a probability, and a classification threshold converts that probability into a label. Lowering the threshold flags more transactions as fraud, which generally increases recall and reduces precision; raising it does the opposite.

Application in Project: multiple classification thresholds were evaluated for the XGBoost model, and **0.25 produced the best F1-score among the thresholds tested**. This does not make 0.25 a universal optimum. It is the best of the tested values for this model and this test period, and a different dataset, period, or business cost of errors could favor another value. The application-defined risk levels (Table 4.2) are based on this threshold.

6.5 Model Evaluation Metrics

Because the fraud class is rare, performance is judged with metrics that focus on the minority class. They are defined using the confusion matrix (Table 6.1), where TP, FP, FN, and TN are the numbers of true positives, false positives, false negatives, and true negatives, and fraud is the positive class.

Table 6.1: Confusion Matrix Layout

	Predicted legitimate	Predicted fraud
Actual legitimate	True negatives (TN)	False positives (FP)
Actual fraud	False negatives (FN)	True positives (TP)

- **Precision:** the proportion of predicted fraud cases that were actually fraud (Equation 6.5).

$$\text{Precision} = \frac{TP}{TP + FP} \tag{6.5}$$

- **Recall:** the proportion of actual fraud cases that were detected (Equation 6.6).

$$\text{Recall} = \frac{TP}{TP + FN} \tag{6.6}$$

- **F1-score:** the harmonic mean of precision and recall (Equation 6.7).

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{6.7}$$

- **ROC-AUC:** the area under the curve of the true positive rate against the false positive rate across all thresholds. It measures how well the two classes are separated (Equation 6.8 defines the false positive rate).

$$\text{FPR} = \frac{FP}{FP + TN} \tag{6.8}$$

- **PR-AUC:** the area under the precision-recall curve. It is particularly informative when the positive class is rare, because it is not inflated by the large number of true negatives.
- **Confusion matrix:** the counts of true positives, true negatives, false positives, and false negatives at a chosen threshold.

CHAPTER 7

RESULTS AND ANALYSIS

7.1 Introduction

This chapter compares the three trained models, Logistic Regression, Random Forest, and XGBoost, on the chronological test set and then evaluates the effect of threshold optimization on the selected XGBoost model. The two sets of results are reported separately so that they are not confused.

7.2 Experimental Setup

- **Training data:** 1,296,675 transactions (7,506 fraudulent).
- **Testing data:** 555,719 transactions (2,145 fraudulent).
- **Input features:** 18 model input features (Table 5.1).
- **Evaluation design:** chronological, with training on January 1, 2019 to June 21, 2020 and testing on June 21, 2020 to December 31, 2020.
- **Class imbalance handling:** class weighting for Logistic Regression, balanced class weights for Random Forest, and `scale_pos_weight` = 171.75 for XGBoost.
- **Evaluation metrics:** Precision, Recall, F1-score, ROC-AUC, and PR-AUC.

The model comparison values in Sections 7.3 to 7.5 were measured at the **default classification threshold of 0.50**.

7.3 Logistic Regression Results

- Precision: 0.2719
- Recall: 0.0853
- F1-score: 0.1299
- ROC-AUC: 0.6305
- PR-AUC: 0.0435

Logistic Regression provided a baseline, but at the evaluated operating point it performed poorly in detecting the minority fraud class: it found only about 8.5% of the fraudulent transactions, and only about 27% of the transactions it flagged were actually fraudulent.

7.4 Random Forest and XGBoost Results

Random Forest

- Precision: 0.9908
- Recall: 0.4033
- F1-score: 0.5732
- ROC-AUC: 0.9789
- PR-AUC: 0.8200

XGBoost

- Precision: 0.9588

- Recall: 0.6834
- F1-score: 0.7980
- ROC-AUC: 0.9949
- PR-AUC: 0.8537

The two models illustrate the trade-off between precision and recall. Random Forest achieved the higher precision at the default threshold, so nearly all of the transactions it flagged were fraudulent, but it detected only about 40% of the actual fraud cases. XGBoost detected a larger proportion of the actual fraud cases (about 68%) with only a small reduction in precision, and it achieved the higher F1-score and PR-AUC.

7.5 Comparison of Models

Table 7.1 compares the three models at the default threshold of 0.50. Accuracy is not reported because it was not part of the verified model results, and, as explained in Section 1.2, it would not be informative for such an imbalanced problem.

Table 7.1: Model Performance Comparison (Default Threshold = 0.50)

Model	Precision	Recall	F1-score	ROC-AUC	PR-AUC
Logistic Regression	0.2719	0.0853	0.1299	0.6305	0.0435
Random Forest	0.9908	0.4033	0.5732	0.9789	0.8200
XGBoost	**0.9588**	**0.6834**	**0.7980**	**0.9949**	**0.8537**

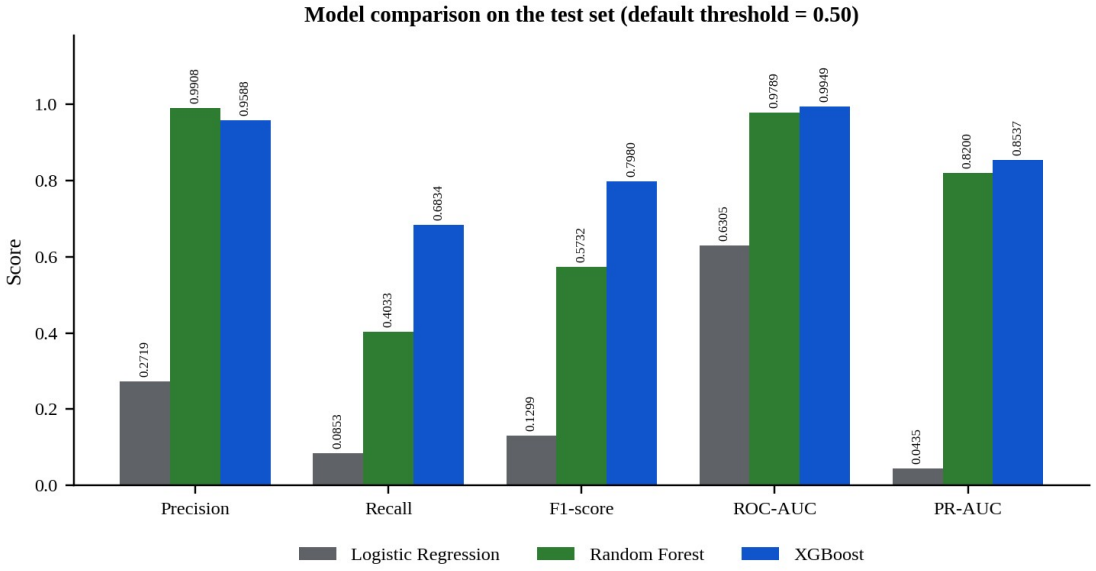


Fig 7.1: Model Performance Comparison

Threshold optimization. The results below are for the selected XGBoost model at the optimized classification threshold of **0.25**. They describe a different operating point and must not be mixed up with the default-threshold XGBoost results in Table 7.1.

Table 7.2: Threshold Optimization Results (XGBoost, Threshold = 0.25)

Measure	Value
Precision	91.53%
Recall	72.54%
F1-score	80.94%
False positives	144
False negatives	589
True positives	1,556
True negatives	553,430

Table 7.3: Test Dataset Confusion Matrix (XGBoost, Threshold = 0.25)

	Predicted legitimate	Predicted fraud	Total
Actual legitimate	553,430	144	553,574
Actual fraud	589	1,556	2,145

At threshold 0.25 the model flagged 1,700 transactions (1,556 true positives and 144 false positives) and missed 589 of the 2,145 fraudulent transactions. Fig 7.2 shows how the precision, recall, and F1-score of XGBoost changed between the default threshold of 0.50 and the selected threshold of 0.25.

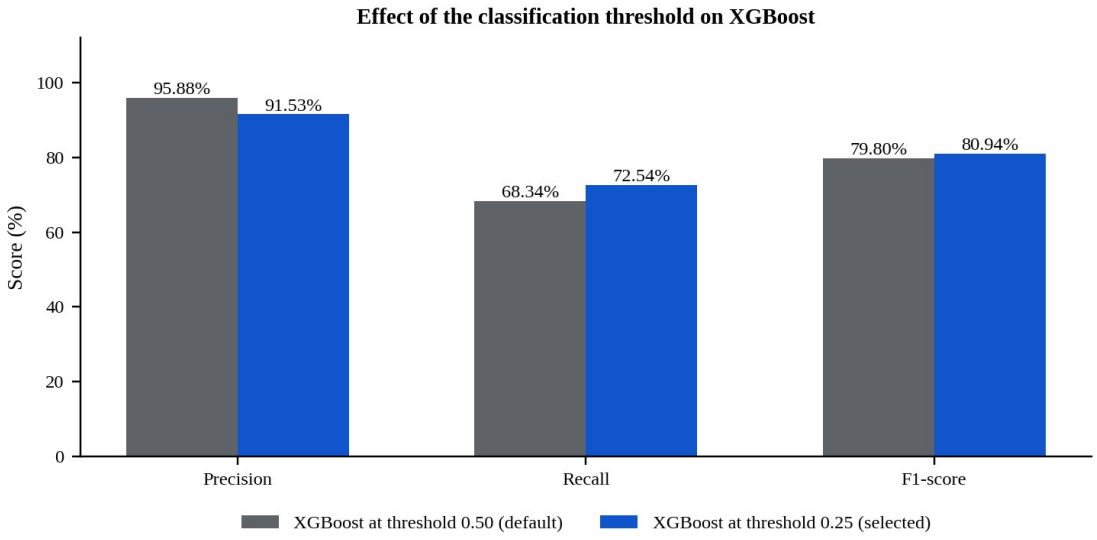


Fig 7.2: Threshold Optimization Results

7.6 Key Findings

- XGBoost achieved the strongest PR-AUC (0.8537) among the three models, along with the highest ROC-AUC and F1-score at the default threshold.
- The F1-score increased from 79.80% at threshold 0.50 to 80.94% at threshold 0.25.
- Recall increased after lowering the threshold (from 68.34% to 72.54%), with a corresponding reduction in precision (from 95.88% to 91.53%).
- The global SHAP summary identified transaction amount, category, previous amount, hour, time since the previous transaction, previous transaction count, amount deviation, and age among the leading features (Fig 7.3).

- The system generates explainable risk estimates, but a model probability does not prove that a transaction is fraudulent.

Fig 7.3 shows the **global** SHAP summary for the model. It describes the overall influence of features across many transactions. Explanations for **individual** transactions are shown separately in Chapter 8.

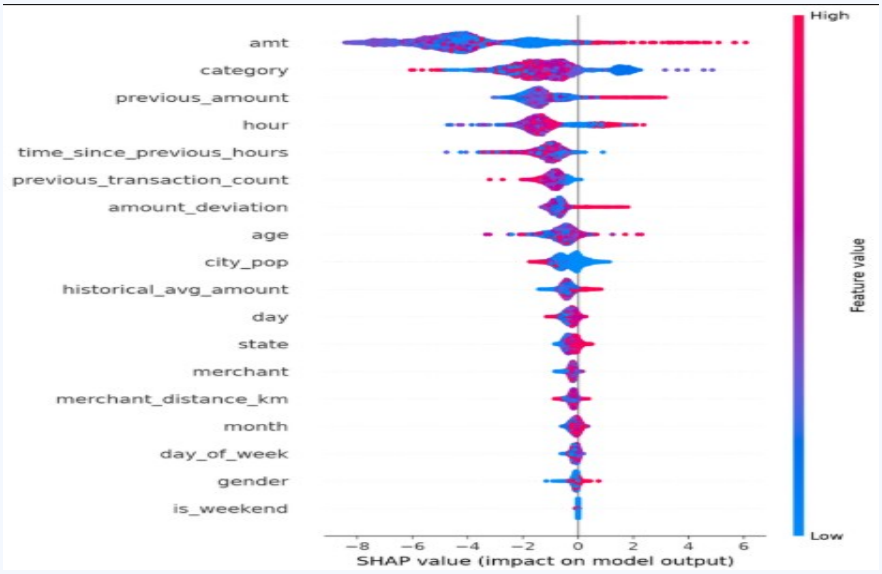


Fig 7.3: SHAP Summary Plot (Global Feature Importance)

CHAPTER 8

OUTPUT SCREENS

This chapter presents the output of the project, from the development environment and model results to the web application. Each frame below is a labeled placeholder for the corresponding screenshot, to be inserted from the actual project outputs. Screenshots must not show raw card numbers or other sensitive transaction identifiers.

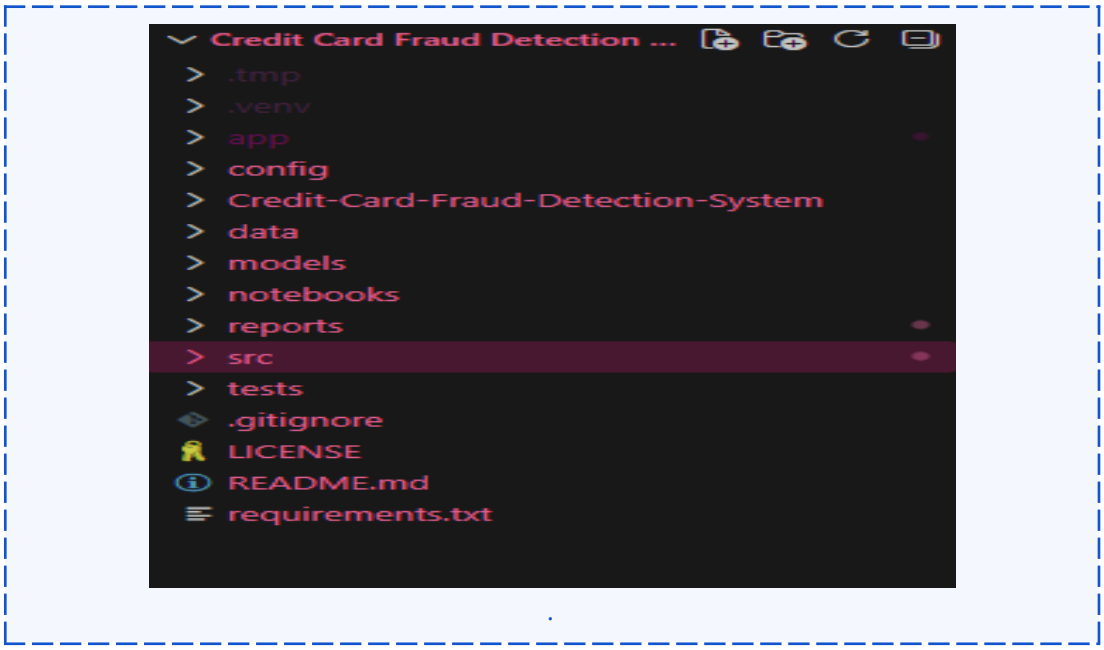


Fig 8.1: Project Directory and Development Environment

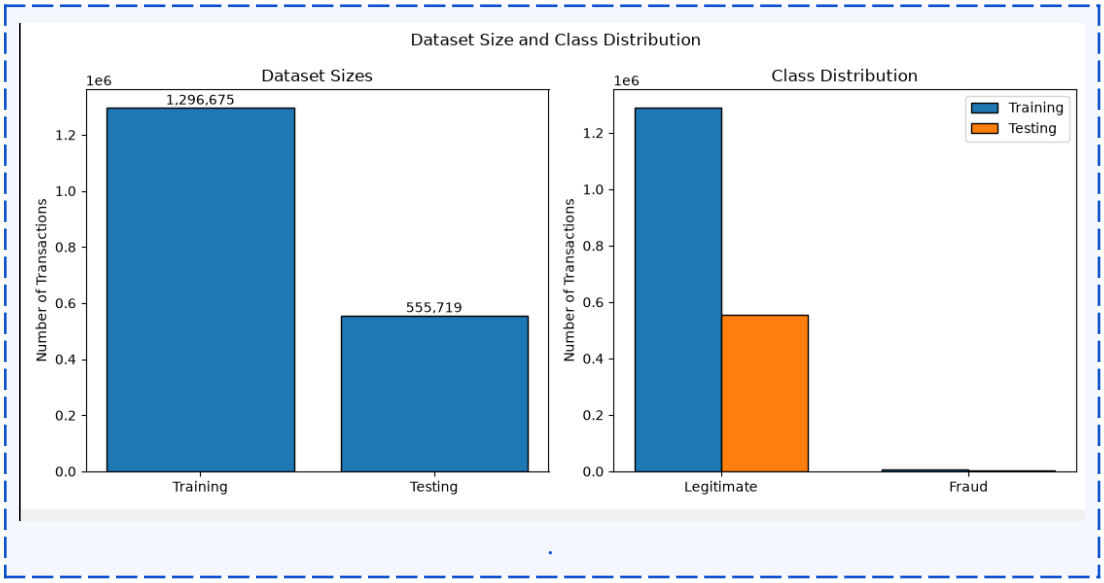


Fig 8.2: Dataset Analysis and Class Distribution Output

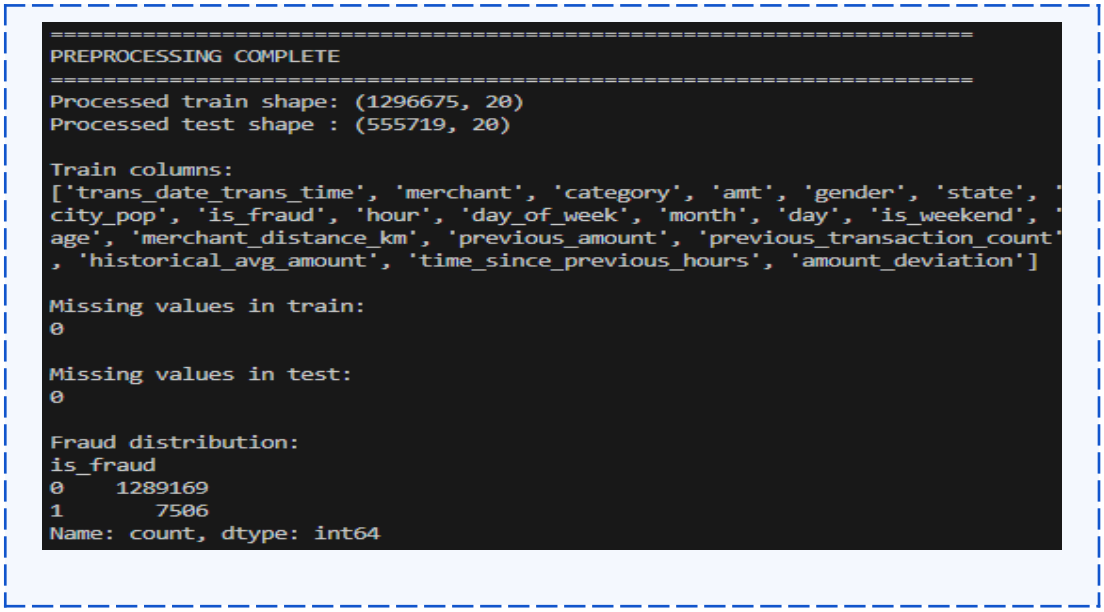


Fig 8.3: Data Preprocessing Output

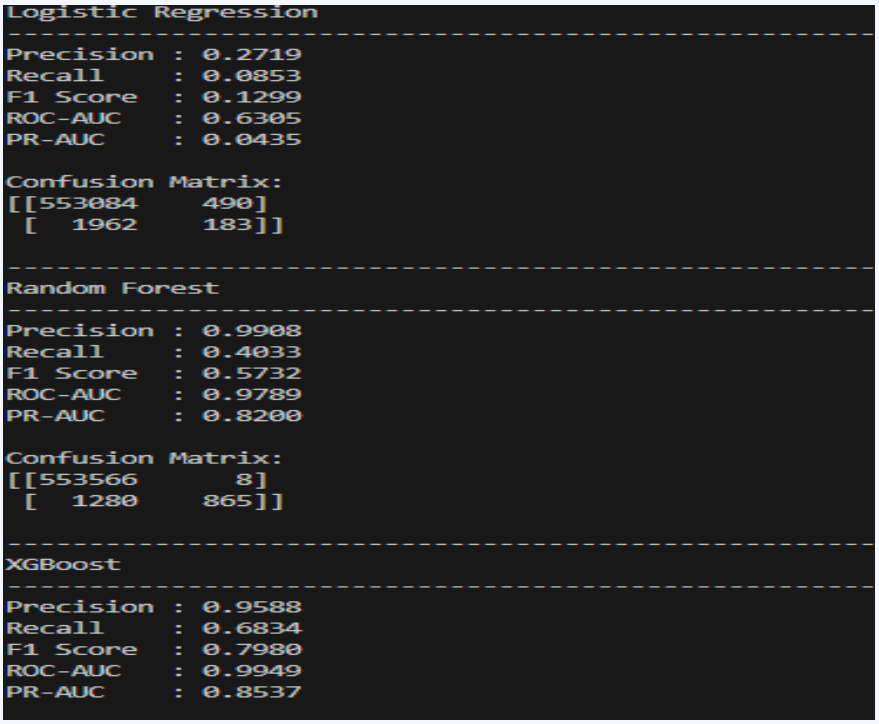


Fig 8.4: Model Training Output

```
================================================================
MODEL COMPARISON
================================================================
              Model  Precision   Recall       F1  ROC-AUC   PR-AUC
Logistic Regression   0.271917 0.085315 0.129879 0.630528 0.043513
      Random Forest   0.990836 0.403263 0.573227 0.978880 0.819959
            XGBoost   0.958797 0.683450 0.798040 0.994892 0.853696
```

Fig 8.5: Model Comparison Table or Chart

```
============================================
BEST F1 THRESHOLD
============================================
Threshold  : 0.25
Precision  : 0.9153
Recall     : 0.7254
F1 Score   : 0.8094
False Positives  : 144
False Negatives  : 589
```

Fig 8.6: Threshold Optimization Results

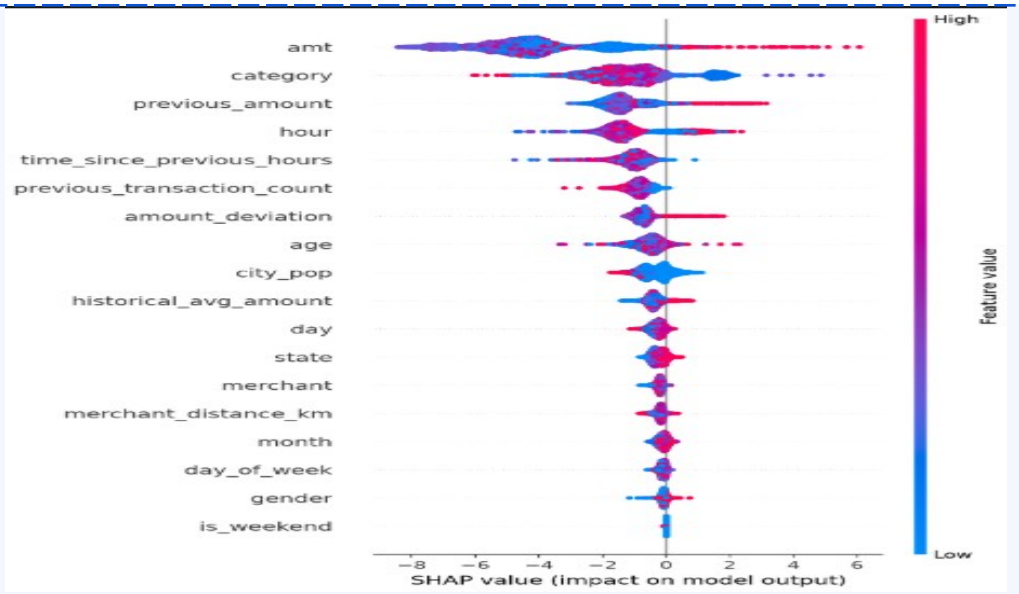


Fig 8.7: SHAP Feature Importance Plot

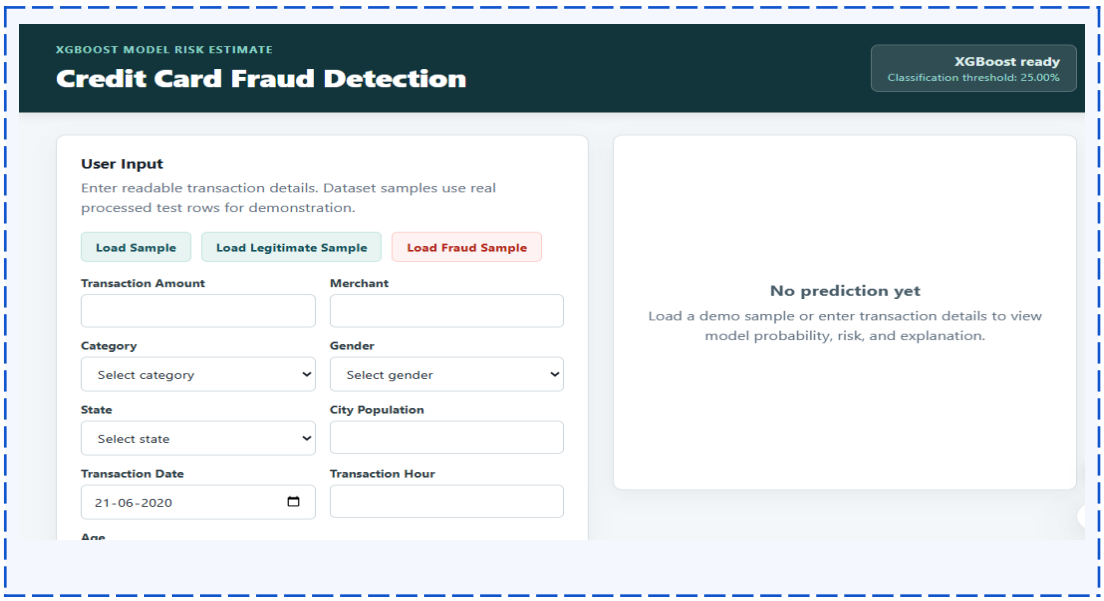


Fig 8.8: Transaction Input Interface

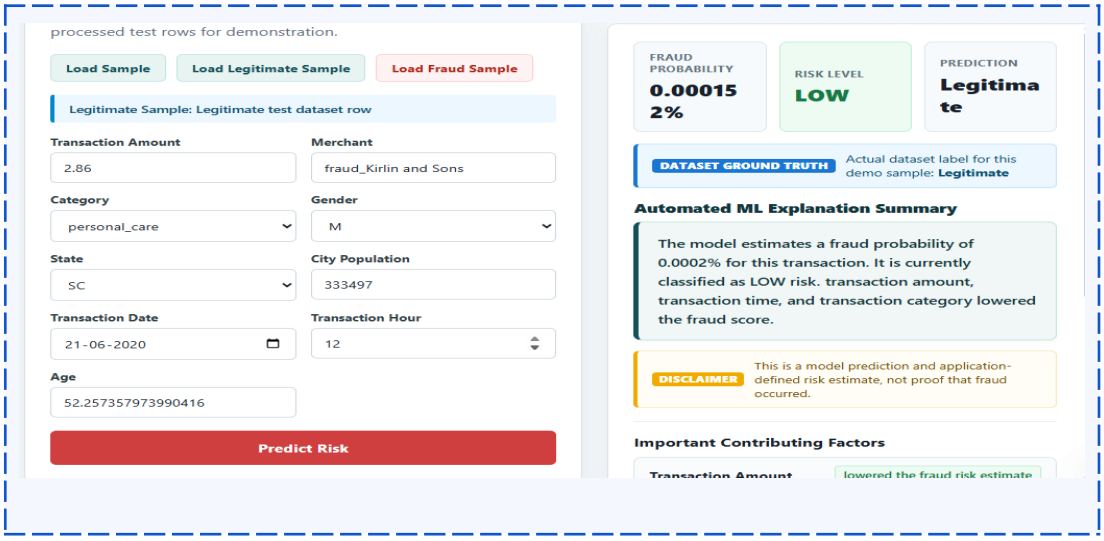


Fig 8.9: Legitimate Sample Prediction

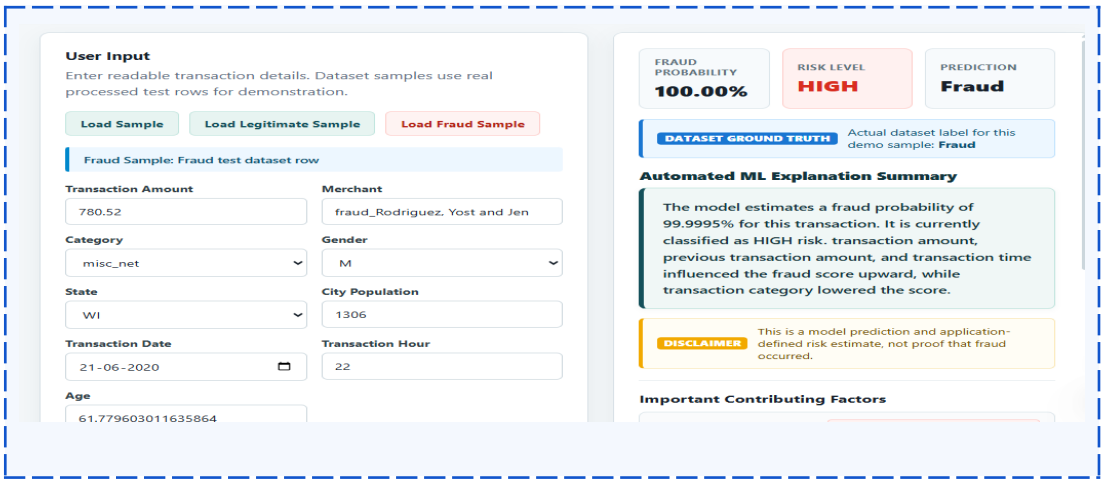


Fig 8.10: Fraud Sample Prediction



Fig 8.11: Fraud Probability and Risk Level

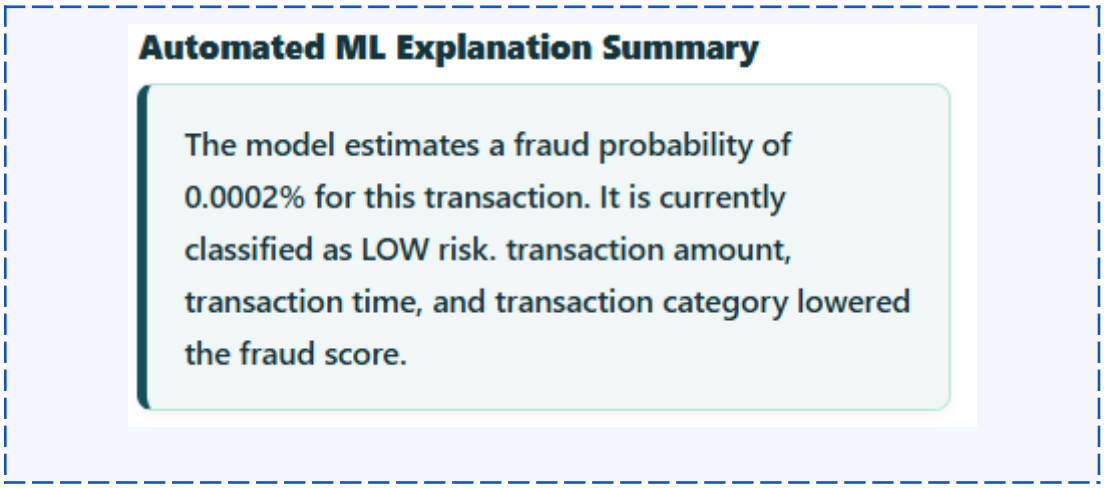


Fig 8.12: Explanation and Contributing Factors

CHAPTER 9

CONCLUSION

This project developed a machine-learning-based credit card fraud detection prototype using historical transaction data and behavioral feature engineering. Starting from the `fraudTrain.csv` and `fraudTest.csv` files, which are severely imbalanced, the data was preprocessed and enriched with transaction-time, geographic, and card-behavior features computed only from information available before each transaction, and models were evaluated chronologically on later transactions. Logistic Regression, Random Forest, and XGBoost were compared, and XGBoost achieved the strongest PR-AUC and F1-score among the tested models at the reported default threshold. Optimizing the classification threshold to 0.25, the best F1-score among the thresholds tested, improved the F1-score to 80.94%, with a precision of 91.53% and a recall of 72.54%, and SHAP provides interpretable explanations of the model's outputs. The integration of the prediction engine with a FastAPI backend and a React frontend demonstrates an accessible way to display probability-based risk estimates together with their explanations. The application is, however, a prototype: it is not connected to a live banking system, it was evaluated on a single dataset, and its predictions are risk estimates that are not proof of fraud.

CHAPTER 10

FUTURE SCOPE

The following are possible future enhancements. They are **not** part of the current implementation.

- **Real-time integration:** connecting the system to authorized real-time transaction streams.
- **Continuous monitoring:** tracking changes in fraud patterns and model performance over time.
- **Periodic retraining:** updating the model with newer data so that it keeps pace with evolving fraud behavior.
- **Improved probability calibration:** making the predicted probabilities better reflect true fraud likelihood.
- **More advanced behavioral features:** adding richer descriptions of cardholder and merchant behavior.
- **Secure transaction-history storage:** storing transaction histories securely to support history-based features.
- **Authentication and role-based access:** controlling who can use the application and what they can see.
- **Scalable deployment:** deploying the application on scalable infrastructure.
- **Analyst feedback and alerting:** letting reviewers confirm or reject predictions and receive alerts, with feedback used to improve the model.
- **Evaluation on additional datasets:** testing the approach on other transaction datasets to check how well it generalizes.

CHAPTER 11

REFERENCES

[1] Kartik Shenoy (Kartik2112). *Credit Card Transactions Fraud Detection Dataset*. Kaggle.

- **Platform:** Kaggle
  Dataset URL: https://www.kaggle.com/datasets/kartik2112/fraud-detection
  Files Used: fraudTrain.csv and fraudTest.csv
  Access Date: 20 September 2026
- The dataset contains simulated credit card transactions generated using the Sparkov Data Generation tool. It includes transaction details and fraud labels for developing and evaluating machine learning models.

[2] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[3] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems 30*, 2017.

[4] XGBoost Developers, "XGBoost Documentation." [Online]. Available: https://xgboost.readthedocs.io

[5] SHAP Developers, "SHAP Documentation." [Online]. Available: https://shap.readthedocs.io

[6] Scikit-learn Developers, "scikit-learn: Machine Learning in Python — Documentation." [Online]. Available: https://scikit-learn.org/stable/

[7] S. Ramírez, "FastAPI Documentation." [Online]. Available: https://fastapi.tiangolo.com

[8] Meta Platforms, Inc., "React Documentation." [Online]. Available: https://react.dev

[9] The pandas Development Team, "pandas Documentation." [Online]. Available: https://pandas.pydata.org/docs/

[10] NumPy Developers, "NumPy Documentation." [Online]. Available: https://numpy.org/doc/

[11] Vite Contributors, "Vite Documentation." [Online]. Available: https://vite.dev

[12] Encode, "Uvicorn Documentation." [Online]. Available: https://www.uvicorn.org. [Accessed: 18-Oct-2026].